

JWT – JSON Web Tokens

A Complete Python Course

From Basics to Production-Ready Implementation

Table of Contents

1. What is JWT?
 2. JWT Structure
 3. Setting Up JWT in Python
 4. JWT with Flask – Full Login System
 5. Refresh Tokens
 6. Security Best Practices
 7. Practical Exercise
 8. Quick Reference Cheat Sheet
-

Module 1: What is JWT?

JWT stands for **JSON Web Token**. It is an open standard (RFC 7519) that defines a compact, self-contained way to securely transmit information between parties as a JSON object. This information can be verified and trusted because it is digitally signed.

[TIP] **Think of JWT like a stamped passport.** When you travel, the passport proves who you are without the border agent calling your home country to verify. The stamp (signature) is trusted globally.

Why Do We Need JWT?

Traditional server sessions store user data on the server. This creates problems at scale:

- Every server must share the same session store
- Memory usage grows with every logged-in user
- Hard to scale horizontally across multiple servers
- Difficult to use across different domains (mobile apps, APIs)

JWT solves this by making the token itself carry the user data. The server does not need to remember anything.

How JWT Works – The 3-Step Flow

Step	Action
Step 1	User logs in with username and password
Step 2	Server creates a signed JWT token containing user info and sends it back
Step 3	Client stores the token and sends it with every future request. Server verifies it without any database lookup.

Module 2: JWT Structure

Every JWT token has exactly **3 parts** separated by dots:

xxxxx.yyyyy.zzzzz

Header.Payload.Signature

Part 1 – Header

The header contains metadata about the token – the type and the signing algorithm.

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

*# This JSON is then Base64URL encoded:
eyJhbGciOiAiSFMyNTYiLCJ0eSI6IkpzZXQ9*

Part 2 – Payload (Claims)

The payload contains the actual data – called **claims**. There are 3 types of claims:

```
# Registered claims (standard, optional but recommended)  
{  
  "sub": "1234567890",    # Subject - who the token is about  
  "iat": 1516239022,     # Issued At - when was it created  
  "exp": 1516242622,     # Expiration - when does it expire  
  "iss": "bridago.com",  # Issuer - who created it
```

```

# Public/Private claims (your custom data)
"user_id": 42,
"email": "john@example.com",
"role": "admin"
}

```

[WARNING] **WARNING:** The payload is only Base64 encoded, NOT encrypted. Anyone can decode it and read the data. **Never put passwords or sensitive secrets in the payload.**

Part 3 – Signature

The signature is what makes JWT secure. It is created by combining the encoded header, encoded payload, and a secret key:

```

HMACSHA256(
    base64UrlEncode(header) + '.' + base64UrlEncode(payload),
    your_secret_key
)

```

Result: SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c

If anyone tampers with the payload, the signature verification will fail and the server will reject the token.

Module 3: Setting Up JWT in Python

We will use the **PyJWT** library – the most popular JWT library for Python.

Installation

```
pip install PyJWT
```

If using Flask

```
pip install Flask PyJWT python-dotenv
```

Your First JWT Token

```

import jwt
import datetime

```

Your secret key -- keep this safe, never commit to GitHub!

```
SECRET_KEY = 'your-super-secret-key-here'
```

```

# Step 1: Create the payload
payload = {
    'user_id': 42,
    'email': 'john@example.com',
    'role': 'admin',
    'iat': datetime.datetime.utcnow(), # issued at
    'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1) # expires in 1hr
}

# Step 2: Encode (create) the token
token = jwt.encode(payload, SECRET_KEY, algorithm='HS256')

print('Your JWT token:')
print(token)
# Output: eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjo0...

```

Decoding and Verifying a Token

```

import jwt

SECRET_KEY = 'your-super-secret-key-here'

token = 'eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...' # token from client

try:
    # Decode and verify in one step
    decoded = jwt.decode(token, SECRET_KEY, algorithms=['HS256'])

    print('Token is valid!')
    print('User ID:', decoded['user_id'])
    print('Email:', decoded['email'])
    print('Role:', decoded['role'])

except jwt.ExpiredSignatureError:
    print('Token has expired! User must log in again.')

except jwt.InvalidTokenError:
    print('Token is invalid! Possible tampering detected.')

```

Module 4: JWT with Flask – Full Login System

Let us build a complete authentication system using Flask and JWT. This is what you would use in a real production API.

Project Structure

```
my_api/  
  app.py  
  auth.py  
  middleware.py  
  .env
```

Step 1 – The .env File

```
SECRET_KEY=replace-this-with-a-very-long-random-string-in-production  
JWT_EXPIRY_HOURS=24
```

Step 2 – Auth Helper (auth.py)

```
# auth.py  
import jwt  
import datetime  
import os  
from dotenv import load_dotenv  
  
load_dotenv()  
SECRET_KEY = os.getenv('SECRET_KEY')  
  
def create_token(user_id, email, role='user'):  
    """Create a new JWT token for a user"""  
    payload = {  
        'user_id': user_id,  
        'email': email,  
        'role': role,  
        'iat': datetime.datetime.utcnow(),  
        'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=24)  
    }  
    return jwt.encode(payload, SECRET_KEY, algorithm='HS256')  
  
def verify_token(token):  
    """Verify a token and return the decoded payload"""
```

```

try:
    decoded = jwt.decode(token, SECRET_KEY, algorithms=['HS256'])
    return decoded, None # return payload and no error
except jwt.ExpiredSignatureError:
    return None, 'Token expired'
except jwt.InvalidTokenError:
    return None, 'Invalid token'

```

Step 3 – Middleware (middleware.py)

Middleware is a function that runs before your route handler to check the token:

```

# middleware.py
from functools import wraps
from flask import request, jsonify
from auth import verify_token

def login_required(f):
    """Decorator to protect routes -- use @login_required"""
    @wraps(f)
    def decorated(*args, **kwargs):
        # Get token from Authorization header
        auth_header = request.headers.get('Authorization')

        if not auth_header:
            return jsonify({'error': 'No token provided'}), 401

        # Header format: 'Bearer eyJ0eXAiOiJKV1Q...'
        try:
            token = auth_header.split(' ')[1] # Get the token part
        except IndexError:
            return jsonify({'error': 'Invalid token format'}), 401

        # Verify the token
        payload, error = verify_token(token)

        if error:
            return jsonify({'error': error}), 401

        # Attach user data to request for use in route
        request.current_user = payload
        return f(*args, **kwargs)

    return decorated

```

```

def admin_required(f):
    """Decorator for admin-only routes"""
    @wraps(f)
    @login_required
    def decorated(*args, **kwargs):
        if request.current_user.get('role') != 'admin':
            return jsonify({'error': 'Admin access required'}), 403
        return f(*args, **kwargs)
    return decorated

```

Step 4 – Main App (app.py)

```

# app.py
from flask import Flask, request, jsonify
from auth import create_token
from middleware import login_required, admin_required

app = Flask(__name__)

# Fake database for this example
USERS = {
    'john@example.com': {
        'id': 1,
        'password': 'password123',    # In real app: use bcrypt to hash!
        'role': 'user'
    },
    'admin@example.com': {
        'id': 2,
        'password': 'adminpass',
        'role': 'admin'
    }
}

@app.route('/login', methods=['POST'])
def login():
    """Public route -- anyone can call this"""
    data = request.get_json()
    email = data.get('email')
    password = data.get('password')

    user = USERS.get(email)

```

```

    if not user or user['password'] != password:
        return jsonify({'error': 'Invalid credentials'}), 401

    # Create and return JWT token
    token = create_token(user['id'], email, user['role'])
    return jsonify({'token': token, 'message': 'Login successful'})

@app.route('/profile', methods=['GET'])
@login_required
def profile():
    """Protected route -- must have valid JWT"""
    user = request.current_user
    return jsonify({
        'user_id': user['user_id'],
        'email': user['email'],
        'role': user['role'],
        'message': 'You are authenticated!'
    })

@app.route('/admin/dashboard', methods=['GET'])
@admin_required
def admin_dashboard():
    """Admin only route"""
    return jsonify({'message': 'Welcome to the admin dashboard!'})

if __name__ == '__main__':
    app.run(debug=True)

```

Testing with cURL

```

# Step 1: Login to get your token
curl -X POST http://localhost:5000/login \
  -H "Content-Type: application/json" \
  -d '{"email": "john@example.com", "password": "password123"}'

# Response:
# {"token": "eyJ0eXAiOiJKV1Q...", "message": "Login successful"}

# Step 2: Use the token to access protected route
curl http://localhost:5000/profile \

```

```
-H "Authorization: Bearer eyJ0eXAiOiJKV1Q..."

# Step 3: Try without token (should fail)
curl http://localhost:5000/profile
# Response: {"error": "No token provided"}
```

Module 5: Refresh Tokens

Access tokens expire quickly (15 minutes to 1 hour) for security. But we do not want users to log in every hour. Refresh tokens solve this.

Access Token = short-lived (15-60 mins), used for every API request.
Refresh Token = long-lived (7-30 days), used ONLY to get a new access token.

Implementing Refresh Tokens

```
# auth.py -- updated with refresh tokens
import jwt
import datetime
import os

SECRET_KEY = os.getenv('SECRET_KEY')
REFRESH_SECRET = os.getenv('REFRESH_SECRET') # different key for refresh tokens!

def create_access_token(user_id, email, role):
    """Short-lived token for API requests (15 minutes)"""
    payload = {
        'user_id': user_id,
        'email': email,
        'role': role,
        'type': 'access',
        'exp': datetime.datetime.utcnow() + datetime.timedelta(minutes=15)
    }
    return jwt.encode(payload, SECRET_KEY, algorithm='HS256')

def create_refresh_token(user_id):
    """Long-lived token to get new access tokens (30 days)"""
    payload = {
        'user_id': user_id,
        'type': 'refresh',
        'exp': datetime.datetime.utcnow() + datetime.timedelta(days=30)
    }
```

```

        return jwt.encode(payload, REFRESH_SECRET, algorithm='HS256')

@app.route('/login', methods=['POST'])
def login():
    # ... verify user credentials ...
    return jsonify({
        'access_token': create_access_token(user_id, email, role),
        'refresh_token': create_refresh_token(user_id)
    })

@app.route('/refresh', methods=['POST'])
def refresh():
    """Get a new access token using refresh token"""
    data = request.get_json()
    refresh_token = data.get('refresh_token')

    try:
        payload = jwt.decode(refresh_token, REFRESH_SECRET, algorithms=['HS256'])

        if payload.get('type') != 'refresh':
            return jsonify({'error': 'Invalid token type'}), 401

        # Issue a fresh access token
        user_id = payload['user_id']
        new_access_token = create_access_token(user_id, email, role)
        return jsonify({'access_token': new_access_token})

    except jwt.ExpiredSignatureError:
        return jsonify({'error': 'Refresh token expired, please login again'}), 401

```

Module 6: Security Best Practices

1. Always Use Strong Secret Keys

```

import secrets

# Generate a strong random secret key
secret_key = secrets.token_hex(64)
print(secret_key)
# Output: a9f3d2e1b8c4f7a2d6e9b1c3f5a8d2e4b7c9f1a3d5e7b9c2f4a6d8e1b3c5f7a9

# Store this in your .env file, NEVER hardcode it in your code

```

2. Hash Passwords – Never Store Plain Text

```
pip install bcrypt
```

```
import bcrypt
```

```
# When user registers -- hash the password
```

```
def hash_password(plain_password):  
    salt = bcrypt.gensalt()  
    hashed = bcrypt.hashpw(plain_password.encode('utf-8'), salt)  
    return hashed
```

```
# When user logs in -- verify the password
```

```
def check_password(plain_password, hashed_password):  
    return bcrypt.checkpw(plain_password.encode('utf-8'), hashed_password)
```

```
# Usage:
```

```
hashed = hash_password('mypassword123')  
print(check_password('mypassword123', hashed))    # True  
print(check_password('wrongpassword', hashed))    # False
```

3. JWT Security Checklist

	Rule	Why
[OK]	Use short expiry times (15-60 min)	Limits damage if token is stolen
[OK]	Use HTTPS always	Prevents token interception
[OK]	Never put passwords in payload	Payload is not encrypted
[OK]	Use different keys for access and refresh tokens	Limits blast radius if key is leaked
[OK]	Store tokens in httpOnly cookies	Prevents XSS attacks from reading tokens
[NO]	Never hardcode SECRET_KEY in code	Could be exposed in version control
[NO]	Never use 'none' algorithm	Allows forging tokens without signature

Module 7: Practical Exercise

Build a complete JWT authentication system for a simple **Todo API**. This exercise covers everything you have learned.

Requirements

- POST /register – create a new user with hashed password
- POST /login – authenticate and return access + refresh tokens
- GET /todos – get user's todos (protected)
- POST /todos – create a new todo (protected)
- POST /refresh – get a new access token
- POST /logout – invalidate the refresh token

Starter Code

```
# app.py -- Complete your implementation
from flask import Flask, request, jsonify
import jwt, bcrypt, datetime, os

app = Flask(__name__)
SECRET = os.getenv('SECRET_KEY', 'dev-secret')

# In-memory storage (use a real database in production)
users = {}
todos = {}
blacklisted_tokens = set() # for logout

@app.route('/register', methods=['POST'])
def register():
    data = request.get_json()
    email = data['email']
    password = data['password']

    if email in users:
        return jsonify({'error': 'User already exists'}), 400

    # TODO: Hash the password and store the user
    # Hint: use bcrypt.hashpw()
    pass

@app.route('/login', methods=['POST'])
def login():
    # TODO: Verify credentials and return JWT token
    pass
```

```

@app.route('/todos', methods=['GET', 'POST'])
def todos_route():
    # TODO: Add @login_required and implement get/create todos
    pass

if __name__ == '__main__':
    app.run(debug=True)

```

Quick Reference Cheat Sheet

```

import jwt, datetime

SECRET = 'your-secret-key'

# CREATE TOKEN
token = jwt.encode({
    'user_id': 1,
    'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1)
}, SECRET, algorithm='HS256')

# VERIFY TOKEN
try:
    data = jwt.decode(token, SECRET, algorithms=['HS256'])
except jwt.ExpiredSignatureError:
    print('Expired')
except jwt.InvalidTokenError:
    print('Invalid')

# PROTECT A FLASK ROUTE
from functools import wraps
from flask import request, jsonify

def login_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        token = request.headers.get('Authorization', '').replace('Bearer ', '')
        try:
            request.user = jwt.decode(token, SECRET, algorithms=['HS256'])
        except:
            return jsonify({'error': 'Unauthorized'}), 401
        return f(*args, **kwargs)

```

```
        return decorated

# USE THE DECORATOR
@app.route('/protected')
@login_required
def protected():
    return jsonify({'user': request.user})
```

[OK] **You are now ready to implement JWT authentication in any Python project.** Remember: keep your secret keys safe, always use HTTPS in production, and never store sensitive data in the JWT payload.